CSC1052 Report

Humayun Awan | humayun.awan2@mail.dcu.ie | 22373636

A4 Idea 1 - ML: Improved Voting Ensemble

For my Machine Learning idea I decided to improve the Voting Ensemble I made in A3. This voting ensemble had 3 Decision Tree models and used a hard/majority voting method. The accuracy of the Decision Trees were all <= 70% and could be improved. The addition of 2 more Machine Learning models would also provide better informed predictions in the Voting Ensemble.

5-model Voting Ensemble:
The first step in improving the accuracy of my Voting Ensemble was the addition of 2 new models to use in predictions. From reviewing my work in A2, I found that the logistic regression and multinomial naive bayes models had the highest accuracy of >80%, and decided to use them. I simply fitted the models in my notebook and added them to my Voting Ensemble function. The accuracy of my Voting Ensemble model increased from 68.92% to 76.72%. This of course meant adding the 2 new models was a success. To further attempt to improve my voting ensemble, I tried increasing the accuracy of the of the individual models inside it.

Improve Individual Models:
To improve the individual models within my Voting Ensemble, I decided to 1. Increase the max_features, and 2. Change the vectorizer to TF-IDF. Before max_features = 1000 i.e. select the top 1000 most frequent words as features, but I decided to increase the number to 2500. The TF-IDF vectorizer is better than CountVectorizer as it weighs the importance of words rather than just having a simple count of how often they appear. These changes improved all but one of my Decision Tree models' accuracy, but did not improve the accuracy of my Voting Ensemble, which remained the exact same. This means that although the models predicted better results they couldn't have been for the same reviews, as the other models would outvote them.

A4 Idea 2 Web App: Implement Voting Ensemble

The first idea for my Web App is to implement the improved Voting Ensemble. To do so I will need the same 2 pickle files as before, as well as a voting_ensemble.py file, to store the Voting Ensemble function. This function works using the following inputs: 1. a review 2. A vectorizer and 3. individual models. It gets these from the 1. user input 2. features.pkl file and 3. models.pkl file. These pickle files are created at the end of my A4 notebook where I proceed to dump the 5 models (models.pkl) and vectorizer (features.pkl) into them. The 5 models are my 3 Decision Tree models (“log_loss”, “entropy”, “gini”), a logistic regression model and a multinomial naive bayes model. The vectorizer I use is the TF-IDF vectorizer, with max_featues equal to 2500. They are later opened in the app.py where it predicts a review using the Voting Ensemble function. The other aspects of the web app remain the same, if a review returns 1 print “Positive” and if a review returns 0 print “Negative”. In my following idea I attempt to improve the User Interface (UI) of the web app by styling the html files.

A4 Idea 3 - Web App: Improve UI

My final idea was to improve the User Interface of my Flask Web Page. After implementing the Voting Ensemble the app still lacked a clean and professional feel. Without changing any of the code surrounding the Voting Ensemble model and its features, I edited the styling of the html pages and felt a huge improvement in the pages’ look. I included the following style changes in my index.html and predict.html files:

Global Styling:
To make the pages easier to read I used an Arial or Helvetica font. The colours included in my Global Styling were the background colour, for a clean look, and a default text colour. Padding was also included in my Global Styling to prevent text from being too close to either the left or right side of the page.

Heading Styling:
Here I simply gave a colour to the main heading of my web page, “Film Review Classifier”. I opted for blue, which stands out among the pages.

Paragraph Styling:
For the text describing my Film Review Classifier I changed the font-size and added space between paragraphs, to ensure better spacing. I also set a colour for the text but this simply stayed as black.

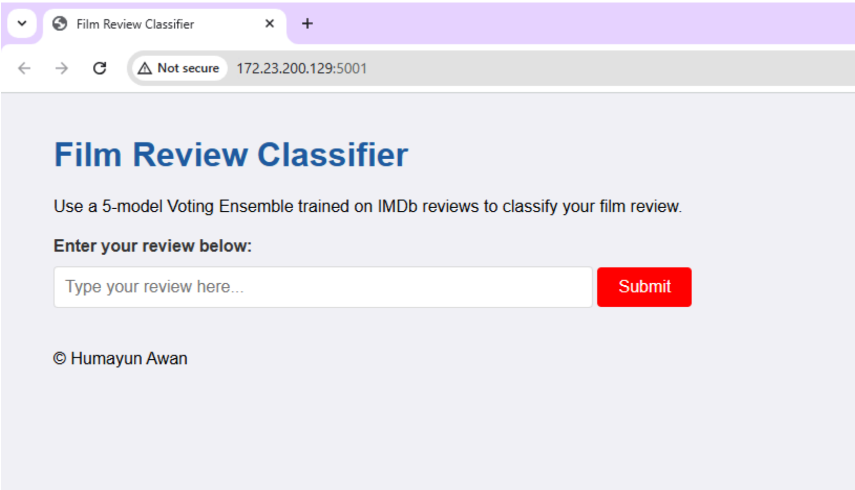
Form Styling:
Here “margin-top” was used to add space above the form, keeping separation between the other text.

Label Styling:
My label is “Enter your review below”, placed above the input field. To highlight it I put it in bold. I made sure the labelled appeared on its own line above the input and ensured space between the two, for better readability.

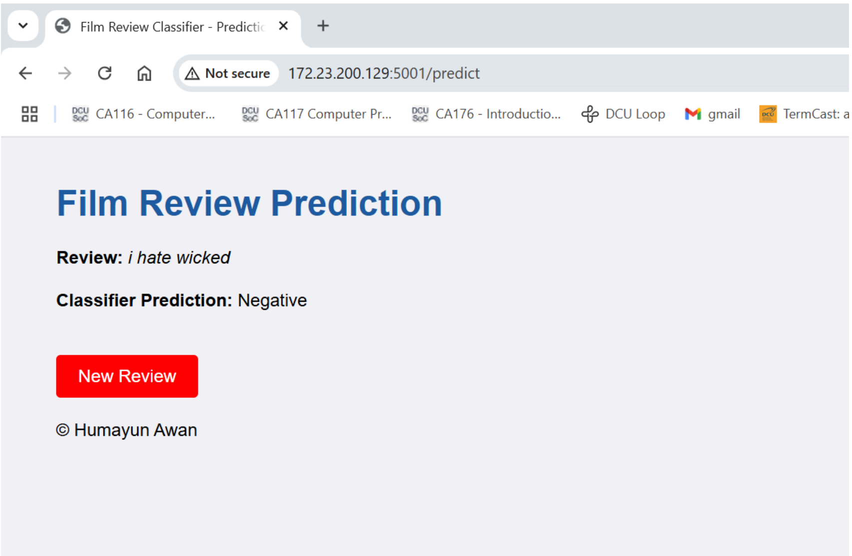
Input Field Styling:
To make sure the input box was not too long, I brought its width down to 20%. I increased the input texts’ font size, and added round corners. Space was also added inside and below the input field, again for better readability.

Button Styling:
I made the button red and its text white, to stand out, and kept space i.e. “padding” inside. For better user interaction I included features which made the cursor a pointer when above the button, and had a transition between the button’s colour when hovered over.

Below is a snippet of the improved home page of the Film Review Classifier web app:



Below is a snippet of the improved prediction page of the Film Review Classifier web app:



Personal Retrospective

In this module I completed 4 assignments and 1 lab exam. In the first assignment I tested the accuracies of different Machine Learning models on sentiment analysis tasks, improving the accuracy of the worst one. This provided a good understanding into sentiment analysis, as I had never encountered it before. In the next assignment I returned the top 50 features of a model from A1, using their importance. I also predicted hard and easy reviews, seeing the breakdown of how a decision is made in LIME. In A3 we made a basic 3-model Voting Ensemble, and created a simple Film Review Classifier web app. In A4 I improved the Voting

Ensemble through adding 2 more individual models, and implemented this into the web app, also improving the app’s UI.